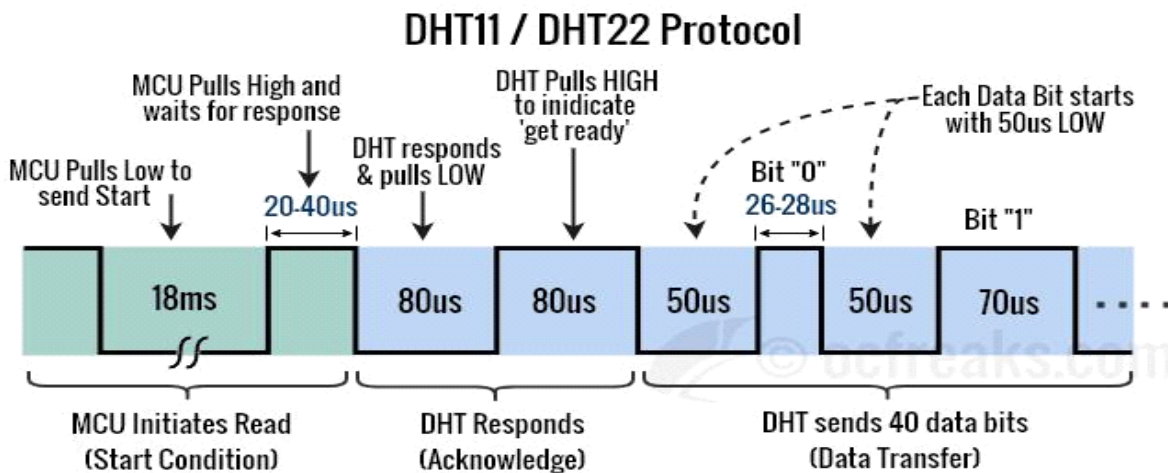


DHT22 sensor working

13 May 2026 15:34



The working of the sensor can be best understood referring the above diagram.

DHT22 Working Principle

The DHT22 sensor has 4 pins:

- VCC
- DATA
- NC (Not Connected)
- GND

The VCC and GND pins are connected to the 3.3V and GND pins of the processor or Raspberry Pi 4 Model B.

The DATA pin is connected to a GPIO pin of the processor.

Communication between the processor and the sensor happens through this single DATA wire, so both the processor and the sensor communicate one at a time.

Inside the DHT22 there is:

- a humidity sensing element
- a temperature sensing element
- a small internal microcontroller

The sensing elements detect changes in electrical properties caused by temperature and humidity. These sensed values are initially analog in nature.

The internal microcontroller reads these analog electrical changes and uses an internal ADC (Analog to Digital Converter) to convert them into digital values.

The processor initiates communication by pulling the DATA line LOW for about 1ms.

Then the processor pulls the line HIGH briefly and changes the GPIO pin to input mode to wait for the sensor response.

The sensor acknowledges the request by:

1. pulling the DATA line LOW
2. then pulling it HIGH

This indicates that the sensor is ready to transmit data.

The sensor then sends 40 bits of data serially through the DATA line.

The 40 bits contain:

- humidity data
- temperature data
- checksum data

Data transmission is completely based on pulse timing.

At the start of every bit:

- the sensor pulls the DATA line LOW for about 50µs

Then the sensor sends the actual bit value by controlling how long the line stays HIGH:

- about 26µs HIGH → bit 0

- about 70µs HIGH → bit 1

The processor continuously measures these HIGH pulse durations to determine whether the transmitted bit is 0 or 1.

Reception at processor

13 May 2026 16:19

How the Processor Receives Data from the DHT22

- > After initiating communication, the processor changes the GPIO pin connected to the DATA line from output mode to input mode.
- > Now the processor no longer drives the DATA line and only monitors the voltage changes on the line.
- > The GPIO input circuitry can detect:
 - LOW voltage (0V)
 - HIGH voltage (3.3V)
- > The processor continuously checks the GPIO input state and observes transitions between LOW and HIGH signals. This process is called edge detection.
- > When the sensor starts transmitting data, each bit transmission begins with:
 - approximately 50µs LOW pulse
- > After this LOW pulse, the sensor drives the DATA line HIGH.
- > The processor waits for the DATA line to become HIGH and immediately starts a timer or counter.
- > The processor continuously monitors the GPIO pin while the line remains HIGH.
- > When the sensor pulls the line LOW again:
 - the processor stops the timer
 - calculates how long the line stayed HIGH
- > The processor determines the transmitted bit based on the HIGH pulse duration:
 - about 26µs HIGH → bit 0
 - about 70µs HIGH → bit 1
- > The processor repeats this process for all 40 transmitted bits. Each received bit is stored and grouped into bytes.
- > The processor then reconstructs the binary data into humidity and temperature values.

For example:

00000010 10001101

is converted into decimal format and interpreted as:

653 → 65.3%

Similarly, the temperature data is reconstructed.

This is how the processor receives and decodes data from the DHT22 sensor.

Pigpio lib

13 May 2026 17:20

pigpio Installation Issue

The normal installation of [pigpio](#) using: `sudo apt install pigpio` failed because the system was running:

- Debian 13 (Trixie)

In this Debian release, the pigpio package is currently not available in the default APT repositories. APT repositories contain precompiled software packages maintained for a particular Linux distribution version. Since the pigpio package was unavailable in the Trixie repository, the package manager could not find an installation candidate.

To solve this issue, pigpio was installed directly from source code.

The installation process involved:

1. cloning the pigpio source repository using git
2. compiling the source code using make
3. installing the compiled binaries and libraries using make install

This method manually builds the library on the target system and is commonly used in embedded Linux environments when prebuilt packages are unavailable.

1. gpiolInitialise()

`gpiolInitialise()`: Initializes the pigpio library and connects the program to the pigpiod daemon. Must be called before any pigpio GPIO operation.

2. gpioTerminate()

`gpioTerminate()`: Stops pigpio operation and releases resources. Usually called at the end of the program.

3. gpioSetMode()

`gpioSetMode(4, PI_OUTPUT);` or

`gpioSetMode(4, PI_INPUT);`

Configures GPIO direction.

The DATA line is shared.

So processor must:

- first become OUTPUT → send start signal
- then become INPUT → receive sensor data

Mode	Meaning
------	---------

PI_OUTPUT	GPIO output
-----------	-------------

PI_INPUT	GPIO input
----------	------------

4. gpioWrite()

`gpioWrite(4, 0);`

or

`gpioWrite(4, 1);`

Drives GPIO voltage level.

5. gpioRead()

`gpioRead(4)`: Reads current voltage level on GPIO pin.

Return	Meaning
--------	---------

0	LOW
---	-----

1	HIGH
---	------

6. gpioDelay()

gpioDelay(1000): Creates delay in microseconds.
1000 microseconds = 1 millisecond

7. gpioTick()

gpioTick(): Returns current system tick count in microseconds.
Used to measure pulse duration.

Example:

```
start = gpioTick();
```

later:

```
duration = gpioTick() - start;
```

This calculates:

How long line stayed HIGH

8. time_sleep()

time_sleep(2): Creates delay in seconds.

Overall DHT22 Program Flow Using pigpio

Initialize pigpio



Set GPIO OUTPUT



Pull line LOW



Delay 1ms



Pull line HIGH



Switch GPIO INPUT



Read pulse timings



Measure HIGH durations



Construct bits



Convert data



Print temperature/humidity



Terminate pigpio

Interface code

13 May 2026 18:15

```
#include <stdio.h>
#include <stdint.h>
#include <pigpio.h>

#define DHT_PIN 4

// Global variables

uint8_t data[5] = {0};

int bit_index = 0;
int started = 0;
int edge_cnt = 0;

uint32_t rising_tick = 0;

// Callback function

void dht_callback(int gpio, int level, uint32_t tick)
{
    // Ignore invalid events

    if(level != 0 && level != 1)
        return;

    // Count valid edges only

    // Rising edge

    if(level == 1)
    {
        rising_tick = tick;
    }

    // Falling edge

    else if(level == 0)
    {
        edge_cnt++;
        uint32_t width;

        width = tick - rising_tick;

        // Ignore initial sensor response edges

        if(edge_cnt == 3)
        {
            started = 1;
            bit_index = 0;

            for(int i = 0; i < 5; i++)
            {
```

```

        data[i] = 0;
    }

    return;
}

// Decode actual data bits

if(started && bit_index < 40)
{
    // Shift previous bits

    data[bit_index / 8] <<= 1;

    // Decode current bit

    if(width > 50)
    {
        data[bit_index / 8] |= 1;
    }

    bit_index++;

    // Full packet received

    if(bit_index == 40)
    {
        uint8_t checksum;

        checksum =
            (data[0] +
             data[1] +
             data[2] +
             data[3]) & 0xFF;

        if(checksum == data[4])
        {
            float humidity;
            float temperature;

            humidity =
                ((data[0] << 8) | data[1]) / 10.0;

            temperature =
                (((data[2] & 0x7F) << 8)
                 | data[3]) / 10.0;

            // Negative temperature

            if(data[2] & 0x80)
            {
                temperature = -temperature;
            }

            printf("\n");

```

```

        printf("Temperature : %.1f C\n",
            temperature);

        printf("Humidity   : %.1f %%\n",
            humidity);

        printf("\n");
    }

    else
    {
        printf("Checksum failed\n");
    }

    // Reset for next reading

    started = 0;
    edge_cnt = 0;
    bit_index = 0;
}
}
}
}
}

```

```

int main()
{
    // Initialize pigpio

    if(gpioInitialise() < 0)
    {
        printf("pigpio initialization failed\n");
        return -1;
    }

    // Register callback

    gpioSetAlertFunc(DHT_PIN, dht_callback);

    while(1)
    {
        // Reset states before new request

        started = 0;
        edge_cnt = 0;
        bit_index = 0;

        // Send start signal

        gpioSetMode(DHT_PIN, PI_OUTPUT);

        gpioWrite(DHT_PIN, 0);

        gpioDelay(1000);

        gpioWrite(DHT_PIN, 1);

        gpioDelay(30);
    }
}

```



```
// Switch to input mode

gpioSetMode(DHT_PIN, PI_INPUT);

// Wait before next reading

time_sleep(2);
}

gpioTerminate();

return 0;
}
```

3V3 (1) (2) 5V
GPIO2 (3) (4) 5V
GPIO3 (5) (6) GND
GPIO4 (7) (8) GPIO14

Errors faced

18 May 2026 10:05

1. pigpio Installation Failed

Error :

Package pigpio is not available

Unable to locate package libpigpio-dev

Reason: The System was Debian, pigpio package was not available in current repository.

Tried: apt update, installing pigpio, installing libpigpio-dev -> still failed.

Final Solution: Built pigpio manually from source using GitHub source code.

2. Linker Errors While Compiling

Error:

undefined reference to gpiointialise
and many pigpio functions.

Reason: Program compiled without linking pigpio library.

Tried: gcc dht22.c

Final Solution: gcc dht22.c -o dht22 -lpigpio

3. Permission Error Running Program

Error: Sorry, you don't have permission to run this program

Reason: pigpio requires root privileges for GPIO access.

Final Solution:

Run using: sudo ./dht22

4. Sensor Returned 0 Values

Error:

Temp : 0.00 C

Humidity : 0.00 %

Reason:

Bits were not being decoded properly.

Polling-based decoding timing unreliable.

Tried:

Moved from:

polling loops to -> pigpio callback-based decoding.

5. Callback Synchronization Errors

Error:

Checksum failed

and garbage values like:

Humidity : 1650%

Temperature : 1645 C

Reason: Synchronization pulses were decoded as actual data bits. This shifted entire 40-bit packet.

Initially synchronization used:

if(width > 150)

to detect frame start.

This proved unreliable.

6. Edge Counting Synchronization Attempt

Switched from: pulse-width sync detection -> edge-count synchronization.

Idea:

ignore first few sensor response edges

before decoding actual data.

7. Continuous Reset Bug

Error:

```
if(edge_cnt > 3)
```

Problem:

After edge count became 4:

- decoding reset continuously
- data array cleared repeatedly
- callback always returned early.

Final Fix: Changed to: `if(edge_cnt == 3)`

so synchronization happened only once.

8. Invalid Level Check Bug

Error: Program printed nothing.

Reason:

Condition used:

```
if(level != 0 || level != 1)
```

which is always true.

Why

For any value:

- one side becomes true
- OR condition always true.

Final Fix

Changed to: `if(level != 0 && level != 1)`

9. Edge Counter Placement Bug

Reason: `edge_cnt++` placed before invalid-event filtering.

So:

- timeout/error events
- non-edge events

also increased edge count.

Final Fix

Moved: `edge_cnt++`;

after:

```
if(level != 0 && level != 1)  
    return;
```